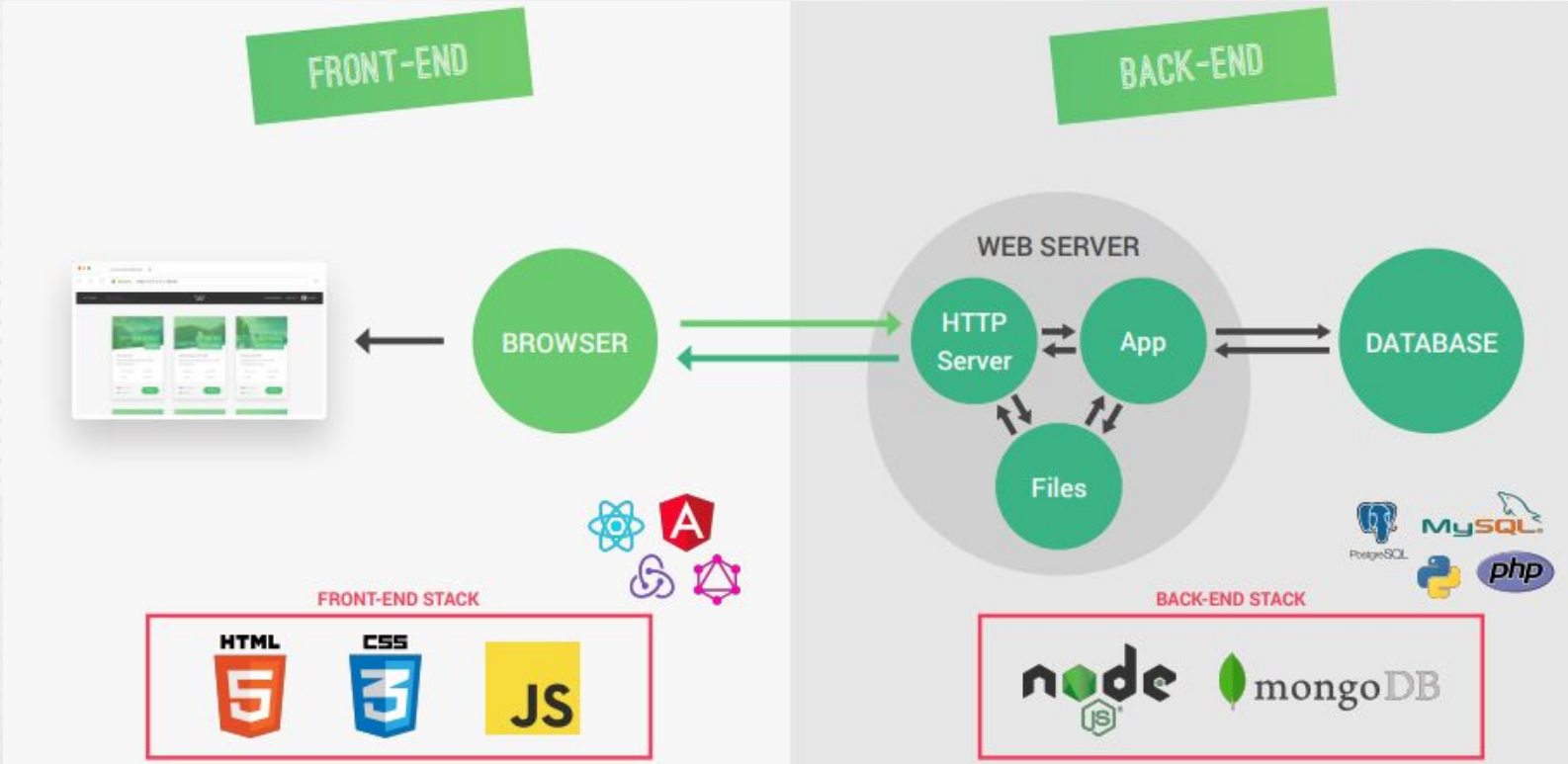
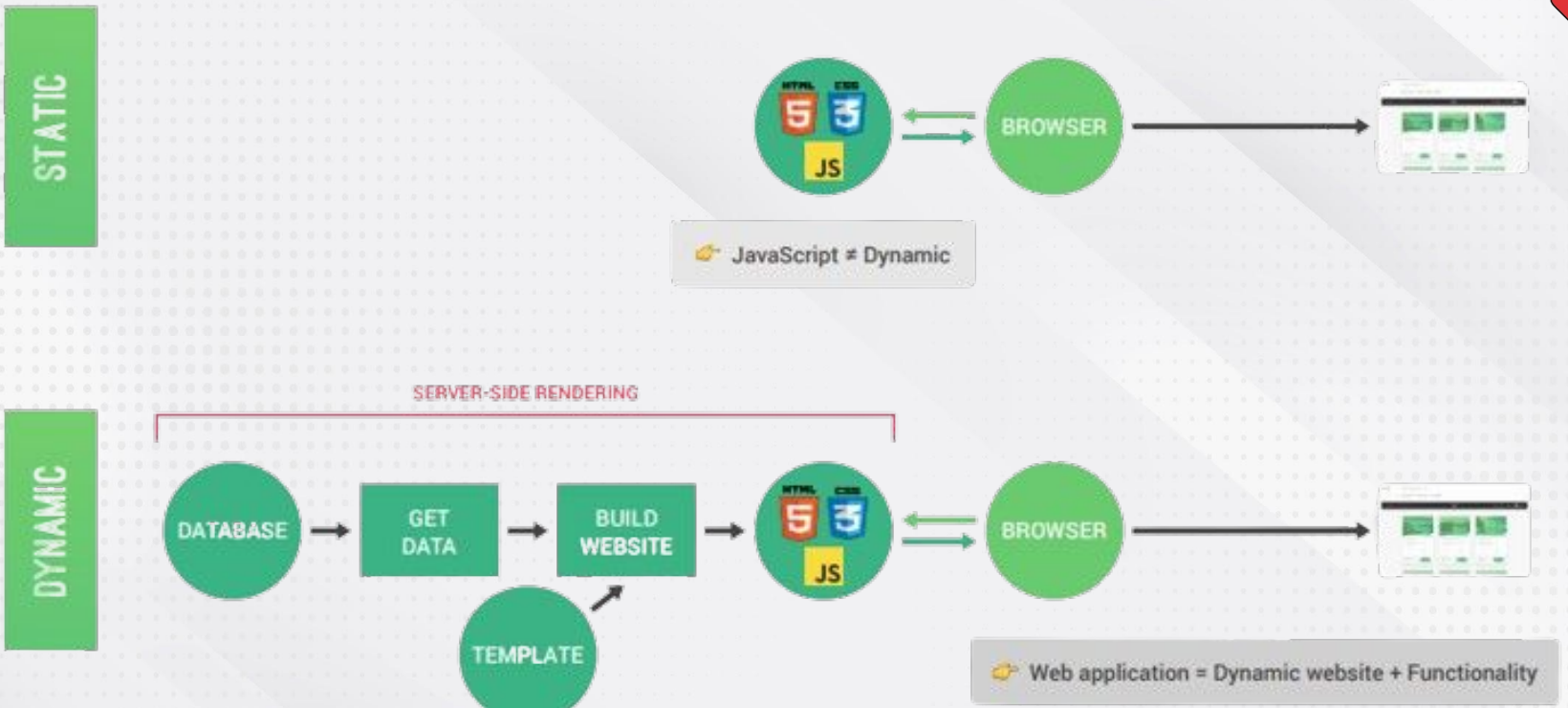

Express


POSTMAN

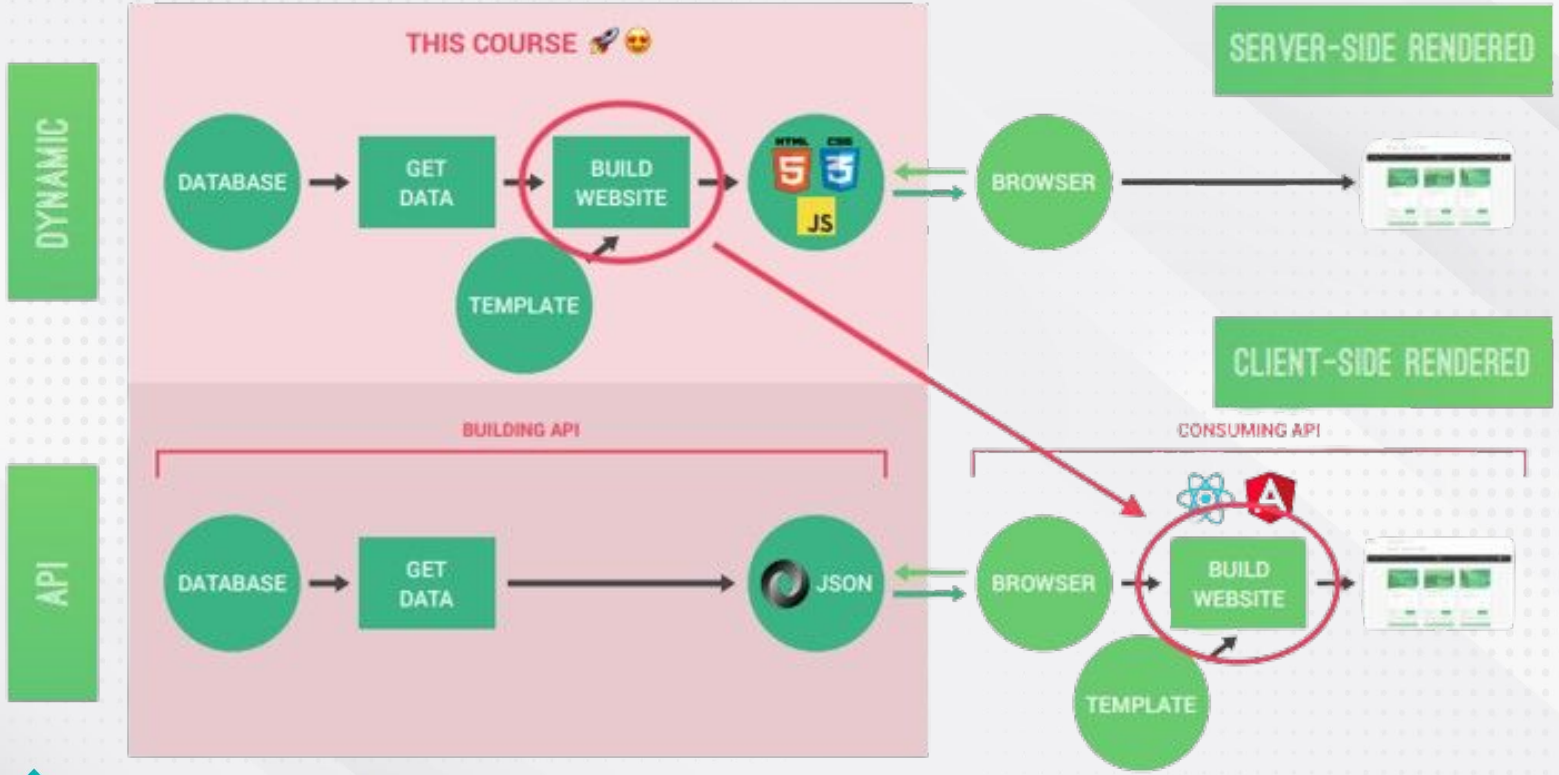
FRONT-END and BACK-END



Static vs Dynamic Website



Dynamic vs API-Powered Website



What and Why Express?

- Express is a minimal node.js framework. It is higher level of abstraction.
- Express contains a very robust set of features: complex routing, easier handling of requests and responses, middleware, server-side rendering, etc.
- Express allows for rapid development of node.js applications.
- Express makes it easier to organize our application into the MVC architecture.

Advantages of Express

- It is simple and lightweight software. It is not heavy to get installed in the machine and make the application running.
- It is easy to customize and configure.
- It is a better choice for creating the API as when the application requires various APIs to communicate with different sources then the Express is a good option.

Postman Introduction

- Postman is a standalone software testing API (Application Programming Interface) platform to build, test, design, modify, and document APIs.
- It is a simple Graphic User Interface for sending and viewing HTTP requests and responses.
- In postman, nearly any functionality that any developer may need is embedded.
- Postman has the ability to make various types of HTTP requests like GET, POST, PUT, PATCH, and convert the API to code for languages like JavaScript and Python.

Installing Postman

- To install postman visit https://www.postman.com/ then click on product and then download postman.

Features of Postman

- Accessibility
- Use Collections
- Test development
- Automation Testing
- Debugging

Configure Express

- Open vs code and then open folder in vs code.
- Initialize the project using command ***npm init.*** It will ask some details and create a ***package.json*** file.
- Now install the express using the command

  `npm install express@4`
- It will install the express inside the ***node_modules*** folder.
- Now create a file app.js that will contain all the configuration of express.

package.json

```
{
  "name": "expressdemo",
  "version": "1.0.0",
  "description": "This is a node and express project",
  "main": "app.js",
  ▷ Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Mohit Kumar",
  "license": "ISC",
  "dependencies": {
    "express": "^4.18.3"
  }
}
```

Configure Express

- Import the express inside ***app.js*** file

```
const express =
require('express');
```

- Now assign the ***express*** function to ***app*** variable.

```
const app = express();
```

- Now create and start the server
- To run the app use ***nodemon app.js***

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.status(200).send('Response from
the express server....');
});

const port = 8080;
app.listen(port, () => {
  console.log(`App started at the port
${port}......`);
});
```

Configure Express (Send JSON Response)

- We can also send the JSON data from the server using ***json*** method.

```
const express = require('express');
const app = express();
app.get('/', (req, res) => {
  res
    .status(200)
    .json({
      message: 'Response from the express server....',
      app: 'express demo',
    });
});
const port = 8080;
app.listen(port, () => {
  console.log(`App started at the port ${port}......`);
});
```

HTTP Status Codes

- **200 OK:** Indicates that the request has succeeded
- **400 Bad Request:** Server could not understand the request.
- **401 Unauthorized:** Requires Authentication
- **403 Forbidden:** Not allowed to access particular functionality
- **404 Not Found:** Server could not found requested resource / invalid endpoint
- **500 Internal Server Error:** Error occurred at server side while processing the request
- **503 Service Unavailable:** Unable to handle request now

HTTP Methods in Express

- In express, HTTP methods are used to define the behavior of a route.
- Following are the most commonly used HTTP methods in express:
  - **app.get():** Used to handle GET requests.
  - **app.post():** Handles POST requests. Commonly used to send the data to server
  - **app.put():** Handles PUT requests, often used to update existing resources.
  - **app.delete():** Handles DELETE requests, used to delete resources.
  - **app.patch():** Handles PATCH requests for partial updates to a resource.
  - **app.all():** Handles all HTTP methods for a given route.
  - **app.use():** Used for middleware functions. It doesn't depend on the HTTP method.

Creating First API (GET Request)

- Read the data from the json file and send the response in the form of json.

```
const express = require("express");
const fs = require("fs");
const app = express();
const tours = JSON.parse(
fs.readFileSync(`${__dirname}/dev-data/data/tours-simple.json`, "utf-8")
);
app.get("/api/v1/tours", (req, res) => {
  res.status(200).json({
    status: "success",
    data: tours,
  });
});
const port = 9090;
app.listen(port, () => {
  console.log(`Server started at port ${port}`);
});
```

Creating First API (POST Request)

- In POST request we send the data from the client to the server.
- Here we will use a middleware to get the data.

  `app.use(express.json());`
- Middleware is a function that can modify the incoming request data.
- Body attribute is inside the middleware.

```
const express = require("express");
const app = express();

app.use(express.json());

app.post("/api/v1/tours", (req, res) => {
  console.log(req.body);
  res.send("Data posted.........");
});

const port = 9090;
app.listen(port, () => {
  console.log(`Server started at port
${port}`);
});
```

Creating First API (POST Request)

- Get the data from the request body using the middleware function.
- Now we will post this data into the given json file.
- For this first get the id of last element in the file and add one to it and create the id of new object that you want to add.

Creating First API (POST Request)

```
const express = require("express");
const fs = require("fs");
const app = express();
app.use(express.json());
const tours = JSON.parse(
fs.readFileSync(`${__dirname}/dev-dat
a/data/tours-simple.json`)
);

app.post("/api/v1/tours", (req, res)
=> {
  const newId = tours[tours.length -
1].id + 1;
  const newTour = Object.assign({ id:
newId }, req.body);
  tours.push(newTour);
  fs.writeFile(

`${__dirname}/dev-data/data/tours-simple.jso
n`,
    JSON.stringify(tours),
    (err) => {
      res.status(201).json({
        status: "Success",
        data: {
          newTour,
        },
      });
    }
  );
});
const port = 9090;
app.listen(port, () => {
  console.log(`Server started at port
${port}`);
});
```

URL Parameters (Get a Single Object)

- To get the single object we will pass the ***id*** of object from the ***URL***.
- After passing the id we can get it inside the route.

```
const express = require("express");

const app = express();

app.get("/api/v1/tour/:id", (req, res) => {
  const id = req.params.id;
  console.log(id);
});

app.listen(8080, () => {
  console.log("server has started at port
8080.....");
});
```

URL Parameters (Get a Single Object)

```
const express = require("express");
const fs = require("fs");
const app = express();
const data =
fs.readFileSync(`${__dirname}/dev-data/da
ta/tours-simple.json`);
const dataObj = JSON.parse(data);

app.get("/api/v1/tour/:id", (req, res) =>
{
  const id = req.params.id * 1;
  const index = dataObj.findIndex((obj)
=> obj.id === id);
  if (index == -1) {
    return res.status(404).json({
      status: "fail",
      message: "Invalid id",
    });
  }
  const oneObj =
dataObj[index];
  res.status(200).json({
    status: "success",
    data: oneObj,
  });
});

app.listen(8080, () => {
  console.log("server has
started at port 8080.....");
});
```

PUT Method(Example)

```
const express = require('express');
const app = express();
app.use(express.json());
const fs = require('fs');
const user =
JSON.parse(fs.readFileSync('files/use
r.json'));

app.put('/user/:id', (req, res) => {
    const id = req.params.id;
    const { name, password } =
req.body;
    if (!user[id]) {
        return
res.status(400).send('can not find
user');
    }
    user[id] = { name,
password };
    const finaluser =
JSON.stringify(user);

fs.writeFileSync('files/use
r.json', finaluser);
res.status(200).json(user);
});

app.listen(8080,'127.0.0.1'
, () => {
    console.log('server
started');
});
```

Creating First API (PATCH Request)

```
const express =
require('express');
const fs = require('fs');
const app = express();
app.use(express.json());
const jsondata =
JSON.parse(fs.readFileSync('files
/student.json'));

app.patch('/student', (req, res)
=> {
  const bodyobj = req.body;
  const index =
jsondata.findIndex((obj) =>
obj.id === bodyobj.id);
  if (index == -1) {
return res.status(404).send('no
record exist');
  }
  jsondata[index] = {
...jsondata[index], ...bodyobj };

fs.writeFileSync('files/student.jso
n', JSON.stringify(jsondata));
  res.status(200).send('updated');
});

app.listen(8080, () => {
  console.log('server
started....');
});
```

Creating First API (DELETE Request)

```
const express = require("express");
const fs = require("fs");
const app = express();
const data =
fs.readFileSync(`${__dirname}/dev-dat
a/data/tours-simple.json`);
const dataObj = JSON.parse(data);

app.delete("/api/v1/tour/:id", (req,
res) => {
  const id = req.params.id * 1;
  if (id > dataObj.length) {
    return res.status(404).json({
      status: "fail",
      message: "invalid id",
    });
  }
```

```
const index = dataObj.findIndex((obj)
=> obj.id === id);
  dataObj.splice(index, 1);
  fs.writeFile(

`${__dirname}/dev-data/data/tours-simpl
e.json`,
    JSON.stringify(dataObj),
    (err) => {
      res.status(200).json({
        status: "success",
        results: dataObj.length,
        data: dataObj,
      }); } );});
app.listen(8080, () => {
  console.log("server has started at
port 8080.....");
});
```

HTTP Method (app.all())

- The ***app.all()*** method in express is used to handle all HTTP methods for a specific route.
- This means that no matter which HTTP request (e.g., GET, POST, PUT etc.) is sent to the route, the callback function associated with ***app.all()*** will execute.

```
const express = require('express');
const app = express();

app.all('/', (req, res) => {
    res.send(`${req.method} method
request`);
})

app.listen(8080,'127.0.0.1', () => {
    console.log('server started');
});
```

HTTP Method (app.use())

- The ***app.use()*** method in express is used to mount middleware functions.
- Middleware functions are functions that execute during the lifecycle of a request and response.
- They can perform operations on the request or response objects, terminate the request-response cycle, or pass control to the next middleware function in the stack.
- It can be used:
  - ***To handle general or common logic (e.g., logging, authentication).***
  - ***To process incoming request data (e.g., parsing JSON or form data).***
  - ***To serve static files.***
  - ***To handle errors.***

HTTP Method (app.use())

```
const express = require('express');
const app = express();
app.use(express.json());

app.use('/user', (req, res, next) => {
    const { name, password } = req.body;
    if (name === 'mohit' && password === 'test@123') {
        next();
    } else {
        console.log(`User name is: ${name} and password: ${password}`);
        res.status(400).send('Invalid username or password');
    }
});
app.post('/user', (req, res) => {
    res.status(200).send('login success');
})

app.listen(8080,'127.0.0.1', () => {
    console.log('server started');
});
```

Parameters in Express

- When working with Postman, parameters can be passed in following ways
  - ***Route Parameters (in the URL path)***
  - ***Query Parameters (in the URL after ?)***
  - ***Body Parameters (sent in the request body)***

Catching Route Parameters

- Route parameters are part of the URL and defined using **:** in the route path. These are accessible via ***req.params***.
- From postman use http://127.0.0.1:8080/user/76 with HTTP method GET

```
const express = require('express');
const app = express();

app.get('/user/:id', (req, res) => {
    console.log(req.params);
    res.send(`${req.method} method
request`);
});

app.listen(8080,'127.0.0.1', () => {
    console.log('server started');
});
```

Catching Query Parameters

- Query parameters are part of the URL, appended after a ?, and accessible via ***req.query***
- From postman use http://127.0.0.1:8080/user?username=test&password=test@123 with HTTP method GET

```
const express = require('express');
const app = express();

app.get('/user', (req, res) => {
    const { username, password } =
req.query;
    res.send(`Username is: ${username}
and password: ${password}`);
});

app.listen(8080,'127.0.0.1', () => {
    console.log('server started');
});
```

Catching Body Parameters

- For body parameters, you need to parse the body.
- Depending on the type of data, use one of the following middleware:
  - **express.json() for JSON data**
  - **express.urlencoded() for form-encoded data**
- From postman use http://127.0.0.1:8080/user with HTTP method POST

```
const express = require('express');
const app = express();
app.use(express.json());

app.post('/user', (req, res) => {
    console.log(req.body);
    const { name, password } = req.body;
    res.send(`User name is: ${name} and
password: ${password}`);
});

app.listen(8080,'127.0.0.1', () => {
    console.log('server started');
});
```

Pug Template in Express

- Pug is a high-performance template engine used to generate HTML dynamically in server-side applications.
- It is known for its clean, indentation-based syntax, which eliminates the need for traditional HTML tags and closing tags, making the templates easier to read and write.
- Following are the features of pug template:
  - **Simplified Syntax:** Pug uses indentation instead of closing tags.
  - **Dynamic Content:** Pug allows embedding variables and expressions directly into templates.
  - **Logic and Control Structures:** You can use conditionals and loops in templates
  - **Reusable Components:** Supports partials, includes, and mixins to create reusable template components.

Steps to Use Pug with Express

- **Install Pug:** Install the Pug package in your project by using ***npm install pug*** command.
- Create a directory named ***views*** that will contain the pug file
- Create a file named ***index.pug*** inside the views folder and put the following code.

```
html
    head
        title= index
    body
        h1 #{heading} //Inline variable interpolation
        p this is #{info}
```

- Now create a express file that will pass the data from the server to the pug template. This file will look like:

Steps to Use Pug with Express


```
const express = require('express');

const app = express();
app.set('view engine', 'pug');
app.set('views', './views');

app.get('/', (req, res) => {
  res.render('index', { index: "test", heading: "This is heading",
info: "This is info" });
});

app.listen(8080, () => {
  console.log('server started....');
});
```

Using Loop in pug Template

- Loops in Pug are written using the ***each*** keyword. They are used to iterate over arrays or objects.

```
html
    head
        title
        link(rel="stylesheet", href="../css/style.css")
    body
        table.customers
            tr
                th Id
                th Name
                th Age
                th Branch
            each item in data
                tr
                    td #{item.id}
                    td #{item.name}
                    td #{item.age}
                    td #{item.branch}
```

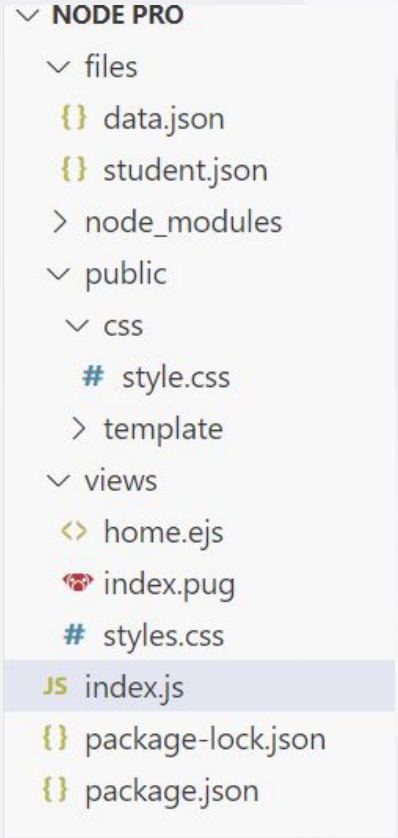
Using Loop in pug Template

- Express file will looks like:

```
const express = require('express');
const fs = require('fs');
const app = express();
app.set('view engine', 'pug');
app.set('views', './views');
app.use(express.static('./public'));

const data =
JSON.parse(fs.readFileSync('files/stud
ent.json'));
app.get('/', (req, res) => {
  res.render('index', { data });
});

app.listen(8080, () => {
  console.log('server started....');
});
```



Serving Static Files in Express

- Static files are the files that are inside our directory and can not be accessed through routes.
- To access the static file in express we use a middleware called ***express.static()***
- In the project create a folder called ***public***, inside this public folder create two subfolders ***css*** and ***templates***.
- Create one css file inside ***css*** folder (***style.css***) and one ***html*** file inside the template folder (***demo.html***).
- If we hit the url http://127.0.0.1:8080/public/templates/demo.html then it will not work.
- To access this html file use:

```
app.use(express.static("./public"));
```

- Now we can access all the static files that are inside the public folder or inside any subfolder of public folder.

Serving Static Files in Express

- Suppose the static file is inside the template folder then on browser we will use `http://127.0.0.1:8080/templates/demo.html`
- We do not need to specify the public folder. Because when we hit the above url from the browser first it will search the route, if the route not found then it will search the static files inside the public folder.

```
const express =
require("express");
const app = express();
app.use(express.static("./public
"));

app.listen(8080, () => {
  console.log("server has
started at port 8080.....");
});
```

Include CSS File in HTML file

- Now to include the css file into the html file use the style tag.

```
<link rel="stylesheet"
href="../css/style.css" />
```

- Now all the styles of ***style.css*** file will be applied to the html.

```
body {
  font-family: "Lucida Sans", "Lucida
Sans Regular", "Lucida Grande",
    "Lucida Sans Unicode", Geneva,
Verdana, sans-serif;
}
h2 {
  color: rgb(216, 215, 213);
}
.main-box {
  border: 1px solid rgb(104, 99, 108);
  text-align: center;
  margin: 50px 200px;
  box-shadow: 0 4px 8px;
}
```

redirect() method in Express

- In Express.js, the ***res.redirect()*** method is used to redirect the client (browser) to a different URL.
- When the server receives a request, it can send a response to the client instructing it to navigate to another URL.
- **Example:**

```
app.get('/', (req, res) => {
  res.redirect('https://www.google.com/');
});
```

Environment Variables

- In case of Node.js we have two main environments: ***development*** and ***production*** environment.
- Based on environment we can on and off certain features like debugging, logging or we can use different database based on environment.
- We can get the environment variable by using `app.get("env")`
- Environment variables are the global variables that are used to define the environment in which node is running.
- All the environment variables of node are inside the ***process*** module. `console.log(process.env);`

Environment Variables

- Install ***dotenv*** by using command ***npm install dotenv.***
- Create a .env file inside the project and put some properties.

```
NODE_ENV=development
STRIPE_SECRET_KEY=sk_test_4eC39HqLyjWDarjtT1zdp7dc
PORT=3000
DEBUG=false
```

- Import dotenv in your project `require('dotenv').config();`
- `console.log(process.env.NODE_ENV);`

REST API

- REST stands for **RE**presentational **S**tate **T**ransfer is an architectural style for distributed hypermedia systems.
- Here we have following terms:
  - ***Resource***
  - ***Representational***
  - ***State***
  - ***Transfer***
- It is one of the most widely used approaches for building web-based APIs.
- REST is not a protocol or a standard, it is an architectural style.
- ***A Web API conforming to the REST architectural style is called a REST API (or RESTful API).***

Six Guiding Principles for REST

- **Uniform Interface:** Resources should be uniquely identifiable through a single URL, and only by using the underlying methods such as DELETE, PUT and GET etc.
- **Client-Server:** UI and request-gathering concerns are the client's domain. Data access, workload management and security are the server's domain. This loose coupling of the client and server enables each to be developed and enhanced independent of the other.
- **Stateless:** All client-server operations should be stateless, and any state management that is required should take place on the client, not the server.

Six Guiding Principles for REST

- **Cacheable:** All resources should allow caching unless explicitly indicated that caching is not possible.
- **Layered System:** REST allows for an architecture composed of multiple layers of servers.
- **Code on Demand (Optional):** Most of the time, a server will send back static representations of resources in the form of XML or JSON. However, when necessary, servers can send executable code to the client.

Middleware in Express

- ***Middleware*** functions are functions that have access to the request object (req), the response object (res), and the next middleware function in the application's request-response cycle. The next middleware function is commonly denoted by a variable named ***next***.
- Middleware functions can perform the following tasks:
  - ***Execute any code.***
  - ***Make changes to the request and the response objects.***
  - ***End the request-response cycle.***
  - ***Call the next middleware function in the stack.***

Types of Middleware in Express

- Application-level middleware
- Router-level middleware
- Error-handling middleware
- Built-in middleware
- Third-party middleware

Add Some Data to Request (Middleware)

```
const express = require("express");
const fs = require("fs");
const app = express();
const tours = JSON.parse(

fs.readFileSync(`${__dirname}/dev-data/data/tours-simple.json`)
);
app.use((req, res, next) => {
  console.log("Hello from middleware");
  next();
});
app.use((req, res, next) => {
  req.reqTime = new Date().toISOString();
  next();
});
```

```
app.get("/tours", (req, res) => {
  res.status(200).json({
    status: "success",
    RequestTime: req.reqTime,
    results: tours.length,
    data: tours,
  });
});
const port = 8080;
app.listen(port, () => {
  console.log(`Server started at port ${port}`);
});
```

Sessions in Express

- HTTP is stateless, in order to associate a request to any other request, you need a way to store user data between HTTP requests.
- Cookies and URL parameters are both suitable ways to transport data between the client and the server. But they are both readable and on the client side.
- Sessions solve exactly this problem. You assign the client an ID and it makes all further requests using that ID. Information associated with the client is stored on the server linked to this ID.
- To use session in express we use external middleware.

  npm install express-session

Sessions in Express


```
const app = express();
const session = require("express-session");

app.use(session({ secret: "My Secret" }));
app.get("/", (req, res) => {
  req.session.username = "This is session username";
  res.send("Response from the server");
});

app.get("/session", (req, res) => {
  res.send(`The session username is ${req.session.username} `);
});
const port = 8080;
app.listen(port, () => {
  console.log(`Server started at port ${port}`);
});
```

Sessions Example to Count User Visit

```
const express = require("express");
const app = express();
const session = require("express-session");
app.use(session({ secret: "My Secret" }));
app.get("/user", (req, res) => {
  if (req.session.userVisit) {
    req.session.userVisit++;
    res.send(`You are visiting this ${req.session.userVisit} times`);
  } else {
    req.session.userVisit = 1;
    res.send("You are visiting this site first time");
  }
});
const port = 8080;
app.listen(port, () => {
  console.log(`Server started at port ${port}`);
});
```